KATEDRA INFORMATIKY
UNIVERZITA PALACKÉHO V OLOMOUCI
www.inf.upol.cz

Michal Krupka
krupka.inf.upol.cz
michal.krupka@upol.cz
17. listopadu 12, 771 46 Olomouc

Paradigmata programování 1 ⋄ poznámky k přednášce

8. Funkce vyššího řádu

verze z 19. listopadu 2024

1 Funkce jako hodnoty

Na minulých přednáškách jsem několikrát upozornil na velkou podobnost některých naprogramovaných funkcí. Ukažme si jeden příklad. Následující funkce k danému seznamu čísel vrací seznam s každým prvkem o jedničku větším:

```
(defun list-inc (list)
  (if (null list)
      '()
    (cons (+ (car list) 1)
          (list-inc (cdr list)))))
```

Test:

```
> (list-inc '(1 2 3 4))
(2 3 4 5)
```

Další funkce počítá desítkový logaritmus každého prvku seznamu a výsledky opět vrací v seznamu:

```
(defun list-log-10 (list)
  (if (null list)
      '()
    (cons (log (car list) 10)
          (list-log-10 (cdr list)))))
```

Test:

```
> (list-log-10 '(1 10 100 1000 10000))
(0.0 1.0 2.0 3.0 4.0)
```

Vidíme, že definice funkcí se velmi málo liší — jen na jednom místě.

Abychom podobnost zdůraznili, definujme dvě pomocné funkce:

```
(defun 1+ (n)
  (+ n 1))

(defun log-10 (x)
  (log x 10))
```

(Funkce `1+` se bude hodit i v budoucnu, stejně jako podobná funkce `1-`; přidáme si ji do jazyka.)

Teď můžeme napsat naše dvě funkce ještě podobněji:

```
(defun list-inc (list)
  (if (null list)
      '()
    (cons (1+ (car list))
          (list-inc (cdr list)))))

(defun list-log-10 (list)
  (if (null list)
      '()
    (cons (log-10 (car list))
          (list-log-10 (cdr list)))))
```

Psát dvě různé funkce, které se liší tak málo jako v tomto příkladě, není vhodné. Problém volá po vytvoření **programové abstrakce**, jak jsme o ní mluvili na druhé přednášce.

Když pomineme rozdíly v našich dvou funkcích zjistíme, že obě dělají následující: *Na každý prvek daného seznamu aplikují nějakou funkci a výsledky vrátí opět v seznamu.*

Rádi bychom tedy jako novou abstrakci napsali funkci, která vytvoří seznam, jehož prvky vznikly aplikací dané funkce na prvky daného seznamu. Aby bylo možné to udělat, musíme umět zadat funkci jako argument jiné funkce. Potřebujeme tedy **pracovat s funkcemi jako s hodnotami**.

Funkce v Lispu jsou hodnoty, tedy, jak víme

- mohou být ukládány do proměnných a složek datových struktur,
- používány jako argumenty funkcí,
- být hodnotami výrazů.

Funkce jako hodnoty Listener zapisuje pomocí špičatých závorek `#< ... >`, které říkají, že o nich nemohl vytisknout všechny informace.

Ke zjištění funkce daného jména slouží speciální operátor `function`, k aplikaci funkce dané jako hodnota používáme funkci `funcall`.

Speciální operátor `function`

K danému symbolu vrátí hodnotu jeho funkční vazby.

```
(function symbol) => funkce
```

Zkratka:

```
#'symbol => funkce
```

Příklad:

```
> (function cons)
#<FUNCTION CONS>
```

Operátor `function` se poměrně často používá, proto místo něj (podobně jako u operátoru `quote`) můžeme používat zkratku:

```
> #'cons
#<FUNCTION CONS>
```

Pokud máme funkci danou jako hodnotu, můžeme ji aplikovat na argumenty pomocí funkce `funcall`:

```
> (bind f #'cons)
#<FUNCTION CONS>

> (funcall f 1 2)
(1 . 2)
```

Použití funkce `funcall`:

```
(funcall fun arg1 ... argn)
```

Aplikuje funkci *`fun`* na argumenty *`arg1 ... argn`*. Jejich počet je libovolný, ale musí odpovídat počtu argumentů přijímaných funkcí *`fun`*. Ještě příklady:

```
> (funcall #'list 1 2 3 4)
(1 2 3 4)

> (funcall #'list #'list #'list)
(#<FUNCTION LIST> #<FUNCTION LIST>)
```

```
> (bind list (funcall #'list #'car #'cdr))
(#<FUNCTION CAR> #<FUNCTION CDR>)

> (first (funcall (second list) list))
#<FUNCTION CDR>
```

Funkce `functionp`
testuje, zda daná hodnota je funkce (typový predikát).

V této části jsme si ukázali různé možnosti používání funkce jako hodnoty, tj. jejich ukládání do proměnných a složek datových struktur a používání jako argumentů i návratových hodnot jiných funkcí. Pokud chceme zdůraznit, že nějaká funkce pracuje s jinými funkcemi jako s hodnotami, říkáme o ní, že je to **funkce vyššího řádu**.

Funkce vyššího řádu

Je to funkce, která je určena k tomu, aby pracovala s jinými funkcemi jako s hodnotami, tj. aby jiné funkce přijímala jako své argumenty případně aby funkce vracela jako svůj výsledek.

2 Mapování

Vraťme se k problému ze začátku přednášky:

```
(defun list-inc (list)
  (if (null list)
      '()
    (cons (1+ (car list))
          (list-inc (cdr list)))))

(defun list-log-10 (list)
  (if (null list)
      '()
    (cons (log-10 (car list))
          (list-log-10 (cdr list)))))
```

Řešení:

```
(defun mapcar-1 (fun list)
  (if (null list)
      '()
```

```
    (cons (funcall fun (car list))
          (mapcar-1 fun (cdr list)))))

(defun list-inc (list)
  (mapcar-1 #'1+ list))

(defun list-log-10 (list)
  (mapcar-1 #'log-10 list))
```

Další příklady:

```
> (bind l1 '(1 2 3 4 5 6))
(1 2 3 4 5 6)

> (mapcar-1 #'- l1)
(-1 -2 -3 -4 -5 -6)

> (mapcar-1 #'1+ l1)
(2 3 4 5 6 7)

> (mapcar-1 #'evenp l1)
(NIL T NIL T NIL T)

> (bind l2 '((a) (b c) (d e f) (g h i j)))
((A) (B C) (D E F) (G H I J))

> (mapcar-1 #'car l2)
(A B D G)

> (mapcar-1 #'length (mapcar-1 #'cdr l2))
(0 1 2 3)
```

3 Funkce vyššího řádu k práci se seznamy

Jednu základní, funkci `mapcar-1`, jsem už ukazoval.Už víme, že úkonu, který se seznamy provádí, říkáme **mapování**. Později ji ještě zobecníme. Další základní úlohy uvedeme teď.

Prohledávání. Funkce `findt` najde první prvek seznamu, který se rovná zadanému prvku. K testování rovnosti prvků použije zadaný predikát. V případě neúspěchu vrací `nil`.

```
(defun findt (x list test)
  (cond ((null list) nil)
        ((funcall test x (car list))
         (car list))
        (t (findt x (cdr list) test))))
```

Testy:

```
> (findt 2 '(1 2 3) #'=)
2

> (findt 4 '(1 2 3) #'=)
NIL

> (findt 'b '(a b c) #'eql)
B

> (findt 2 '(1 2 3) #'>)
1

> (findt 2 '(1 2 3) #'<)
3
```

Funkce `find-if` najde první prvek seznamu splňující daný predikát. V případě neúspěchu vrací `nil`.

```
(defun find-if (pred list)
  (cond ((null list) nil)
        ((funcall pred (car list)) (car list))
        (t (find-if pred (cdr list)))))
```

Test:

```
> (find-if #'oddp '(2 4 6 7 10))
7
```

Filtrace. Funkci `remove`, která z daného seznamu vypustí všechny výskyty daného prvku, už známe:

```
CL-USER 57 > (remove 'a '(a b r a k a d a b r a))
(B R K D B R)
```

To ještě není funkce vyššího řádu, ale následující funkce, vytvořené podobně jako dříve, už ano:

```
(defun removet (x list test)
  (cond ((null list) '())
        ((funcall test x (car list))
         (removet x (cdr list) test))
        (t (cons (car list)
                 (removet x (cdr list) test)))))

(defun remove-if (pred list)
  (cond ((null list) '())
        ((funcall pred (car list)) (remove-if pred (cdr list)))
        (t (cons (car list) (remove-if pred (cdr list))))))
```

Testy:

```
> (removet 3 '(1 2 3 4 5 6) #'<)
(1 2 3)

> (remove-if #'evenp '(1 2 3 4 5 6))
(1 3 5)
```

Redukce (též akumulace). Následující funkce vyššího řádu je díky své univerzálnosti použitelná v mnoha situacích:

```
(defun foldr (fun list init)
  (if (null list)
      init
    (funcall fun (car list) (foldr fun (cdr list) init))))
```

Rozmanitost použití funkce je vidět z příkladů:

```
> (foldr #'+ '(1 2 3 4) 0)
10

> (foldr #'+ '(1 2 3 4) 10)
20

> (foldr #'* '(2 4 5) 1)
40

> (foldr #'cons '(a b c d) '())
(A B C D)
```

```
> (foldr #'cons '(a b c d) '(e f g h))
(A B C D E F G H)

> (foldr #'list '(a b c d) '(e))
(A (B (C (D (E)))))

> (foldr #'append-2 '((a b) (c d e) (f)) '())
(A B C D E F)
```

Jak můžeme vidět, funkci `foldr` můžeme použít ke zjednodušení definic některých funkcí. Například

```
(defun append-2 (list1 list2)
  (foldr #'cons list1 list2))

(defun my-remove (x list)
  (labels ((rem (el accum)
             (if (eql x el) accum (cons el accum))))
    (foldr #'rem list '())))
```

Vidíme také, že funkci `foldr` můžeme použít k sečtení nebo vynásobení všech prvků daného seznamu.

4 Explicitní aplikace funkcí `apply`

Během vyhodnocování seznamu, jehož první prvek je jméno funkce, dochází k aplikaci této funkce na seznam argumentů, které vzniknou vyhodnocením prvků *cdr* vyhodnocovaného seznamu. Funkce `apply` umožňuje aplikovat zadanou funkci na seznam argumentů vzniklý až během práce programu:

```
> (apply #'+ '(1 2 3 4))
10

> (apply #'cons '(1 2))
(1 . 2)
```

Funkce přijímá i více než dva argumenty. Pokud je aplikována na více argumentů, tak argumenty předcházející poslednímu (kromě prvního, kterým je vždy aplikovaná funkce) s posledním argumentem spojí do seznamu, takže např. výrazy `(apply #'+ '(1 2 3))`, `(apply #'+ 1 '(2 3))`, `(apply #'+ 1 2 '(3))`, `(apply #'+ 1 2 3 '())` povedou ke stejnému výsledku.

Příklady:

```
> (apply #'min 5 2 '(6 1 7))
1

> (apply #'* 10 (mapcar #'1+ (list 0 1 2)))
60
```

Funkce `apply` je tedy aplikována na následující argumenty: funkci, libovolný počet hodnot a seznam. Výsledkem aplikace je výsledek aplikace zadané funkce na uvedené hodnoty a prvky uvedeného seznamu.

5 Obecné mapování a další příklady

Obecná verze funkce `mapcar` připouští i více než jeden seznam. Obecně ji lze použít takto:

```
(mapcar fun list1 list2 ... listn)
```

Funkce ***fun*** pak musí přijímat n argumentů. Jak funkce `mapcar` pracuje v tomto obecném případě, vidíme z příkladů:

```
CL-USER 27 > (mapcar #'+ '(1 2 3 4) '(5 6 7 8))
(6 8 10 12)

CL-USER 28 > (mapcar #'cons '(1 2 3 4) '(5 6 7 8))
((1 . 5) (2 . 6) (3 . 7) (4 . 8))

CL-USER 29 > (mapcar #'* '(1 2 3 4) '(5 6 7 8) '(9 10 11 12))
(45 120 231 384)

CL-USER 30 > (mapcar #'list '(1 2 3 4) '(5 6 7 8) '(9 10 11 12))
((1 5 9) (2 6 10) (3 7 11) (4 8 12))
```

Jedním z těžších problémů je tuto funkci naprogramovat. Řešení:

```
(defun mapcar (function list1 &rest lists)
  (if (null list1)
      '()
    (cons (apply function
                 (mapcar-1 #'car (cons list1 lists)))
          (apply #'mapcar
                 function
                 (cdr list1)
                 (mapcar-1 #'cdr lists)))))
```

Pomocí funkce `mapcar` lze zjednodušit mnohé funkce, které jsme si už napsali na práci se seznamy. Například funkci `sum-lists-2` na součet dvou seznamů jako vektorů jsme na šesté přednášce napsali takto:

```
(defun sum-lists-2 (list1 list2)
  (if (null list1)
      '()
    (cons (+ (car list1) (car list2))
          (sum-lists-2 (cdr list1) (cdr list2)))))
```

Teď ji můžeme zjednodušit:

```
(defun sum-lists-2 (list1 list2)
  (mapcar #'+ list1 list2))
```

Zobecnění funkce `sum-lists-2` na součet libovolného nenulového počtu vektorů (nenulového proto, že u nulového počtu bychom nevěděli, kolik prvků má výsledný seznam nul mít) pomocí funkce `foldr`:

```
(defun sum-lists (list1 &rest lists)
  (foldr #'sum-lists-2 lists list1))
```

Totéž pomocí funkce `mapcar` a `apply`:

```
(defun sum-lists (list1 &rest lists)
  (apply #'mapcar #'+ list1 lists))
```

Jako jednu úlohu k šesté přednášce jste programovali funkci `scalar-product` na výpočet skalárního součinu dvou vektorů zadaných seznamy. Řešení mohlo vypadat třeba takto:

```
(defun scalar-product (list1 list2)
  (if (null list1)
      '()
    (+ (* (car list1) (car list2))
       (scalar-product (cdr list1) (cdr list2)))))
```

Se znalostí obsahu této přednášky funkci můžeme přepsat:

```
(defun scalar-product (list1 list2)
  (foldr #'+ (mapcar #'* list1 list2) 0))
```

Jak zjednodušit funkci na vynásobení všech prvků seznamu daným číslem? Také pracuje se seznamem jako s vektorem a je funkci `sum-lists-2` značně podobná. Zjednodušit ji stejným způsobem ale nejde:

```
(defun scale-list (list factor)
  (if (null list)
      '()
    (cons (* factor (car list))
          (scale-list (cdr list) factor))))
```

Jde to ale udělat pomocí lokální funkce:

```
(defun scale-list (list factor)
  (labels ((prod (x) (* x factor)))
    (mapcar #'prod list)))
```

Pokud si ale zkusíme nakreslit prostředí, která vznikají při aplikaci funkce `mapcar`, zjistíme, že nevíme, proč funkce `scale-list` vlastně funguje! Analýza problému nás vede k tomu, že je třeba správně rozhodnout, v jakém prostředí se při aplikaci lokální funkce vyhodnocuje její tělo. Podrobné řešení této otázky nechám na příští přednášku. Na této přednášce budeme zatím lokální funkce používat bez znalosti toho, co přesně se při jejich aplikaci děje.

Nové symboly

speciální operátor: `function`

funkce: `funcall`, `apply`

Otázky a úkoly na cvičení

1. Inspirujte se funkcemi vyššího řádu z této přednášky a napište funkce `membert` a `member-if` (funkci `member` znáte z přednášky 5):

```
> (member 'b '(a b c))
(b c)

> (membert 1 '(1 2 3) #'<)
(2 3)

> (member-if #'evenp '(1 2 3))
(2 3)
```

(do jazyka)

2. Podobně napište funkce `countt` a `count-if`. *(do jazyka)*

3. Vylepšete funkci `merge-sort` tak, aby uživatel mohl zadat predikát na porovnávání: místo pevně zadané funkce `<=` by mohl použít například funkci `>=` a tím setřídit čísla od největšího po nejmenší. *(do jazyka)*

4. Pomocí funkcí z této přednášky napište funkci `arithmetic-mean`, která vypočítá aritmetický průměr libovolného nenulového počtu čísel.

5. Zobecněte funkci `equal-lists-p` z cvičení k šesté přednášce, aby přijímala libovolný nenulový počet argumentů.

6. Přepište funkci `length` pomocí `foldr`.

7. Zkuste pomocí funkce `foldr` zjednodušit definici funkce `mapcar-1`.

8. Napište funkci `foldl`, která se chová stejně jako funkce `foldr`, ale daný seznam prochází zleva doprava. Budete potřebovat pomocnou funkci. Napište ji jako lokální funkci pomocí operátoru `labels`. Než funkci `foldl` začnete psát, odhadněte, co by mělo být hodnotou tohoto výrazu:

```
(foldl #'revcons '(1 2 3 4 5) '())
```

kde funkce `revcons` je definována takto:

```
(defun revcons (x y)
  (cons y x))
```

(do jazyka)